

PROGRESSIFY

LifeRPG mobile app that gamifies everyday tasks

XP · Levels · Streaks · Skills

FRONTEND

Bartosz Lubański

BACKEND

Adrian Oryński

Kotlin

Jetpack Compose

Firebase Firestore

Firebase Auth

MVVM

Android Studio

ADD TASK DIALOG — CONTENT

POST A BOUNTY

Task Title *

Title is required

Description

CATEGORY

Spellcraft

Taskforge

Bard's Delight

Scholars' Sa...

Cycle of Ord...

Bodyforge

DIFFICULTY

Easy

Medium

Hard

TIME WINDOW *

Start

📅 Date

🕒 Time

End

📅 Date

🕒 Time

Set start and end date & time

RECURRENCE

None

Daily

Weekly

Selected days

M

CANCEL

CONFIRM

WHAT THE DIALOG CONTAINS

01

Task title and description

02

Category — pick from 6 skills (3×2 grid)

03

Difficulty: Easy / Medium / Hard

04

Time window — start and end date and time

05

Recurrence: Daily / Weekly / Monthly / Selected Days...

!

Validation — inline errors on title and time

The composable is stateless — it takes `onConfirm` as a lambda. All data reaches the `ViewModel` only after the user confirms.

ADD TASK DIALOG — BUTTON LOGIC



CODE SNIPPET — ONCONFIRM (AddTaskDialog.kt)

```
// AddTaskDialog.kt - CONFIRM button
Button(
    onClick = {
        val startTs = buildTimestamp(startDate, startHour, startMinute)
        val endTs    = buildTimestamp(endDate, endHour, endMinute)
        titleError = title.isBlank()
        timeError  = startTs == null || endTs == null

        if (!titleError && !timeError) {
            onConfirm(
                title, desc, selectedCat, selectedDiff,
                startTs!!, endTs!!,
                RecurrenceRule(
                    type           = recurrenceType,
                    selectedDays = selectedDays.toList().sorted(),
                    interval      = interval
                )
            )
        }
    }
)
```

Validation works inline — `titleError` and `timeError` are Compose states that instantly highlight invalid fields without closing the dialog.

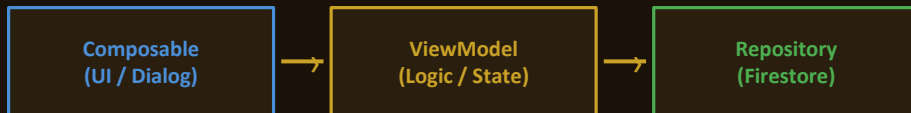
ADD TASK DIALOG — VIEWMODEL



VIEWMODEL RECEIVES DATA (TaskViewModel.kt)

```
// TaskViewModel.kt:106
fun addTask(
    title: String, desc: String, category: String,
    difficulty: String, startTime: Timestamp,
    endTime: Timestamp, recurrence: RecurrenceRule
) {
    val task = Task(
        uid      = uid,    title      = title,
        category  = category, difficulty = difficulty,
        startTime = startTime, endTime   = endTime,
        recurrence = recurrence
    )
    taskRepository.addTask(task) { created ->
        if (created != null) loadTasks()
    }
}
```

DATA FLOW



Unidirectional flow — the UI never writes directly to the database.

DATABASE STRUCTURE — FIRESTORE TREE

COLLECTION TREE

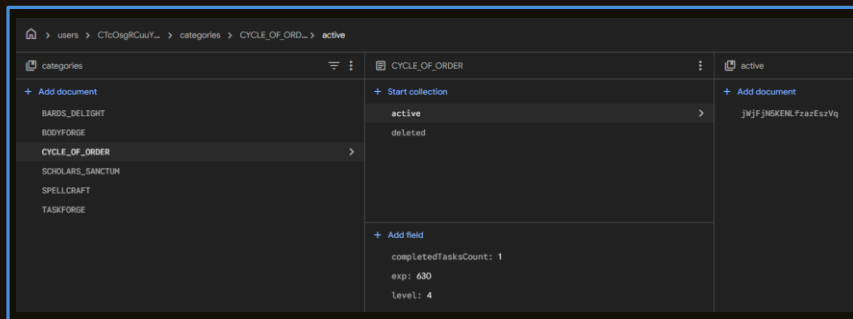
```
users/  
└─ {uid}/  
  │ experiencePoints: Int  
  │ level: Int  
  │ currentStreak: Int  
  │ longestStreak: Int  
  │ completedTasksCount: Int  
  │ streakDates: List<String>  
  │  
  └─ categories/  
    └─ CYCLE_OF_ORDER/  
      │ level: Int  
      │ exp: Int  
      │ completedTasksCount: Int  
      │  
      └─ active/  
        │  
        │ └─ {taskId} → Task  
        └─ completed/  
          │  
          │ └─ {taskId} → Task (isCompleted=true)  
        └─ deleted/  
          │  
          │ └─ {taskId} → Task (kopia)
```

WHY THIS STRUCTURE?

SUBCOLLECTIONS instead of arrays — Firestore can't query fields nested inside arrays. The active / completed / deleted subcollections allow filtering without fetching the whole document.

SOFT-DELETE — deleted tasks move to deleted/ instead of being permanently erased. Data stays safe and can be restored later.

CATEGORIES as user subcollections — each of the 6 skills has its own document with stats (level, exp, count) and task subcollections.



DATABASE STRUCTURE — TASK.KT MODEL

DATA CLASS TASK

```
data class Task(  
    val id          : String          = "",  
    val uid          : String          = "",  
    val title        : String          = "",  
    val category     : String          = "",  
    val difficulty   : String          = "MEDIUM",  
    val startTime    : Timestamp?      = null,  
    val endTime      : Timestamp?      = null,  
    val isCompleted : Boolean           = false,  
    val xpAwarded    : Int              = 0,  
    val recurrence   : RecurrenceRule = RecurrenceRule()  
)
```

DATA CLASS RECURRENCE RULE

```
data class RecurrenceRule(  
    val type          : RecurrenceType = NONE,  
    val selectedDays : List<Int>       = emptyList(),  
    val interval      : Int            = 1  
)  
  
enum class RecurrenceType {  
    NONE, DAILY, WEEKLY,  
    MONTHLY, SELECTED_DAYS  
}
```

Default values in the data class allow direct deserialization from Firestore without mappers — the Firebase SDK fills fields from the document, the rest take their defaults.

`RecurrenceRule.SELECTED_DAYS` lets you pick specific days of the week (e.g. Mon, Wed, Fri). `selectedDays` stores the day numbers as a `List<Int>` sorted ascending.

REPOSITORY — addTask()

FIRESTORE TRANSACTION (TaskRepository.kt:270)

```
// Task.kt:270
fun addTask(task: Task, onComplete: (Task?) -> Unit) {
    val catKey = TaskCategory.entries
        .firstOrNull { it.label == task.category }?.name
    val taskRef = activeColl(task.uid, catKey).document()
    val taskWithId = task.copy(id = taskRef.id)
    val catRef = categoryDoc(task.uid, catKey)

    db.runTransaction { tx ->
        val snap = tx.get(catRef)
        if (!snap.exists()) tx.set(catRef, CategoryStats())
        tx.set(taskRef, taskWithId)
    }.addOnCompleteListener { result ->
        onComplete(if (result.isSuccessful) taskWithId else null)
    }
}
```

WHY A TRANSACTION?

The transaction guarantees atomicity: if the category document doesn't exist yet — it's initialized in the same write as the task. You can't have a task without a category document.

runTransaction() locks the documents during execution and automatically retries on conflict. Safe for concurrent writes from multiple devices.

activeColl() and categoryDoc() are repository helper methods that return references to the relevant Firestore collections for a given user and category.

REPOSITORY — deleteTask()

BATCH WRITE — SOFT-DELETE (TaskRepository.kt:288)

```
// Task.kt:288
fun deleteTask(uid, taskId, category,
               onComplete: (Boolean) -> Unit) {
    val activeRef = activeColl(uid, catKey).document(taskId)
    val backupRef = deletedColl(uid, catKey).document(taskId)

    activeRef.get().addOnSuccessListener { doc ->
        val task = doc.toObject(Task::class.java)

        val batch = db.batch()
        batch.set(backupRef, task)    // copy to deleted/
        batch.delete(activeRef)       // remove from active/
        batch.commit()                // atomically
        .addOnCompleteListener { onComplete(it.isSuccessful) }
    }
}
```

BATCH WRITE VS TRANSACTION

BATCH WRITE — two operations (set + delete) in a single network request. Either both run or neither — no risk of data loss if the connection drops.

A batch doesn't need a read before writing (unlike a transaction), so it's faster when the data is already known — here we have it from an earlier `get()`.

If the task isn't in active/, the code checks completed/ — completed tasks can be soft-deleted too.

completeTask() — THE HEART OF THE XP SYSTEM

ONE TRANSACTION — 3 ATOMIC OPERATIONS (Task.kt:336)

```
// Task.kt:336
db.runTransaction { tx ->
    val task = tx.get(activeRef).toObject(Task::class.java)
    val stats = tx.get(catRef).toObject(CategoryStats::class.java)
    val xp    = task.calculateXp(Timestamp.now())

    // 1. Move: active/ → completed/
    tx.set(completedRef, task.copy(
        isCompleted = true, completedAt = now, xpAwarded = xp))
    tx.delete(activeRef)

    // 2. Update category stats
    val newExp = stats.exp + xp
    val newLevel = computeLevel(newExp)
    tx.set(catRef, CategoryStats(newLevel, newExp, stats.completedTasksCount + 1))

    // 3. Global counter (server-side)
    tx.update(userRef, "completedTasksCount",
        FieldValue.increment(1))

    xp // return xp as the transaction result
}
```

WHY A TRANSACTION? It eliminates the "read-then-write" race condition. XP and level stay consistent, even on a weak connection or with concurrent operations.

FieldValue.increment(1) — the server-side increment doesn't need to read the user document inside the transaction. Faster and safe for concurrent writes.

Three operations in one transaction: moving the task, updating category stats and incrementing the global counter — all or nothing.

completeTask() — XP CALCULATION



XP CALCULATION FORMULA (Task.kt:85)

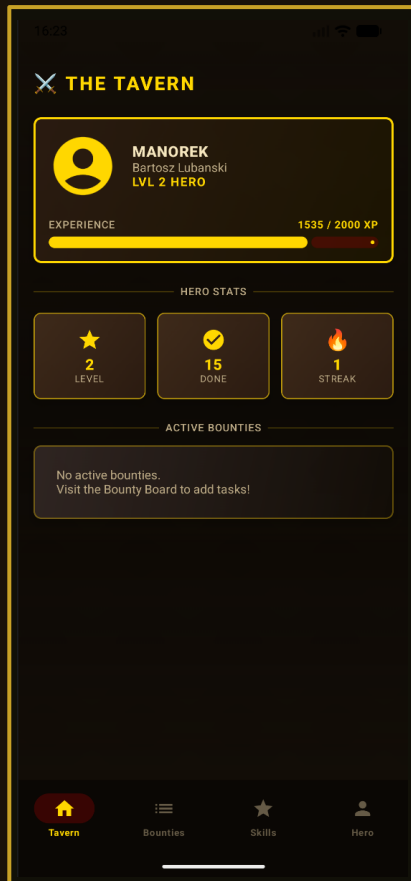
```
// Task.kt:85 — calculateXp()
val base    = difficulty.xpBase    // Easy=50 Med=100 Hard=200
val scaler  = durationMinutes / 30f
val ratio    = getTimeDiff(now)    // spent/allocated

val xp = round(base * scaler * ratio / 10) * 10
```

CALCULATION EXAMPLE

Difficulty	Hard
base XP	200
Task duration	60 min
scaler	2.0
Completed in ½ the time	ratio = 0.5
XP result	200 XP

SUMMARY — 6 WEEKS



IMPLEMENTED MECHANICS

- ✓ Authentication (Firebase Auth)
- ✓ Task system: CRUD + soft-delete
- ✓ Dynamic XP calculation
- ✓ Global hero level
- ✓ 6 independent skill levels
- ✓ Streak system with date tracking
- ✓ Recurring tasks (auto-creation)

WHAT'S NEXT — WEEKS 7-10

Character classes

Statistics

UI Polish

E2E Tests